

1. WHAT WE NEED FOR PROCTOEASE (COMPLETE INVENTORY)

Think of this project as **5 layers**. Every item below is **mandatory or strongly justified** by your research.

A. HUMAN & KNOWLEDGE PREREQUISITES (YOU / TEAM)

1. Core Technical Skills (Must-Have)

You (or your team) must be comfortable with:

Frontend

- JavaScript (ES6+)
- React (hooks, context, state management)
- Browser APIs:
 - MediaDevices API (camera/mic)
 - Page Visibility API
 - Web Audio API
 - Service Workers
 - IndexedDB
- Basic WebRTC concepts (media streams)

Backend

- Python 3.10+
- FastAPI (async/await, dependencies, routers)
- REST + WebSocket APIs
- JWT authentication & OAuth2
- Role-Based Access Control (RBAC)

Databases

- PostgreSQL
- SQL schema design
- Row-Level Security (RLS)
- Indexing & transactions

DevOps

- Docker & Docker Compose
 - Git & GitHub
 - Basic Linux commands
 - CI/CD concepts
-

2. Conceptual / Academic Prerequisites (Very Important for MCA)

You must **understand and be able to explain**:

- Human-in-the-loop AI
- Explainable AI (XAI)
- Heuristic risk models vs black-box ML
- Multi-tenant SaaS architecture
- Offline-first systems
- Data protection laws (DPDP Act, GDPR basics)
- AI bias & fairness

👉 **Good news:** your uploaded PDFs already cover these deeply. You are not missing theory.

2. SOFTWARE, TOOLS & TECHNOLOGIES REQUIRED

A. FRONTEND STACK (EXACT)

Core

- **React** (Vite or CRA)
- **Monaco Editor** (for coding exams)
- **Tailwind / MUI** (optional, UI polish)

Browser APIs (CRITICAL)

Used directly, not optional:

- `navigator.mediaDevices.getUserMedia()`
- Page Visibility API (`document.visibilitychange`)

- Web Audio API (AudioContext + AnalyserNode)
- IndexedDB (via idb or Dexie.js)
- Service Workers
- Background Sync API

📌 These APIs are the **core innovation** of ProctoEase.

B. AI MODELS – WHAT, WHERE, HOW

1. Face Detection & Gaze Tracking (CLIENT-SIDE)

Option 1 (Recommended)

MediaPipe

- What it gives:
 - Face detection
 - Facial landmarks
 - Head pose
 - Gaze direction (via landmarks)
- Runs **inside browser (WASM)**
- Extremely lightweight

Where to get it

- Official docs: <https://developers.google.com/mediapipe>
- JS package: [@mediapipe/face_mesh](#)

Option 2 (Backup)

face-api.js

- Built on TensorFlow.js
- Good for face detection & recognition
- Slightly heavier than MediaPipe

Where

- <https://github.com/justadudewhohacks/face-api.js>

✅ Your research strongly prefers **MediaPipe**.

2. Object Detection (Optional but Advanced)

- **YOLOv5 / YOLOv8 (tiny models)**
- Used for:
 - Mobile phone detection
 - Book detection

⚠ This is **Phase 4+**, not MVP-critical.

3. No Training Required

Important:

- **You do NOT train models**
- You use **pre-trained models**
- This keeps:
 - Cost low
 - Bias manageable
 - System explainable

This is academically correct and startup-feasible.

C. BACKEND STACK (MANDATORY)

Core

- **FastAPI**
- **Uvicorn / Gunicorn**
- **Pydantic**
- **SQLAlchemy / SQLModel**
- **asyncpg**

Async Processing

- **Celery**
- **Redis**

Used for:

- Plagiarism checks
 - Snapshot processing
 - Risk score aggregation
 - Report generation
-

D. CODE EXECUTION ENGINE

Judge0 (Non-Negotiable)

What it does

- Secure sandbox for running candidate code
- Supports 60+ languages
- Prevents:
 - Infinite loops
 - Memory attacks
 - OS access

Where

- Open source: <https://github.com/judge0/judge0>
- API docs: <https://ce.judge0.com/>

Why Judge0

- Academically accepted
 - Startup-friendly
 - No vendor lock-in
-

E. PLAGIARISM DETECTION TOOLS

1. JPlag (Primary Engine)

- AST-based
- Ignores:
 - Variable names
 - Comments
 - Formatting

Where

- <https://github.com/jplag/JPlag>

2. MinHash + LSH (Scalability Layer)

- Fast clustering of similar submissions
- Used before JPlag for large batches

Libraries

- Python: `datasketch`
 - Custom implementation possible
-

3. DATABASE & MULTI-TENANCY REQUIREMENTS

Database

- PostgreSQL 14+

Architecture

- Pool model with `tenant_id`
- Row-Level Security (RLS)

Every table includes:

`tenant_id UUID NOT NULL`

FastAPI sets:

`SET app.current_tenant_id = '<tenant_uuid>';`

Postgres enforces isolation automatically.

📌 This is one of your **strongest architectural points**.

4. SECURITY & LEGAL PREREQUISITES

Security

- JWT (access + refresh)
- OAuth2 Password Flow
- RBAC (Student / Recruiter / Admin)
- TLS everywhere
- AES-256 at rest

Legal / Ethical

- Explicit consent screens
- Data minimization
- Snapshot-based proctoring (no continuous video)
- Automated data deletion
- Human review before punishment

Your project **explicitly avoids “AI judge” behavior**, which examiners love.

5. COMPLETE FEATURE LIST (WHAT PROCTOEASE WILL HAVE)

A. CANDIDATE FEATURES

Exam Experience

- Secure login
- Exam instructions + consent
- MCQs, subjective, coding exams
- Monaco-based code editor
- Timer with autosave

Proctoring (Client-Side)

- Face detection
- Multiple-face detection
- Gaze tracking
- Tab-switch detection
- Audio spike detection
- Snapshot capture on violations

Offline-First

- Exam continues without internet
 - Answers saved in IndexedDB
 - Automatic sync on reconnect
 - No panic / no data loss
-

B. RECRUITER / ADMIN FEATURES

Exam Management

- Create exams
- Question banks
- Time windows
- Threshold rules

Proctoring Dashboard (XAI)

- Risk score (0–100)
- Interactive timeline
- Event-by-event explanation
- Snapshot evidence
- Gaze plots (advanced)

Evaluation

- Auto grading (MCQ)
 - Code execution results
 - Plagiarism reports
 - Manual override allowed
-

C. SYSTEM FEATURES

Multi-Tenant SaaS

- Multiple institutions
- Full data isolation
- Tenant-level RBAC

Automation

- Zero-touch filtering
- Auto shortlist
- Notifications
- Export reports

Sustainability

- No continuous video streaming
- 90% bandwidth reduction
- 96% lower carbon footprint

- SDG 4, 9, 10, 13 aligned
-

6. WHAT YOU DO NOT NEED (IMPORTANT)

You **do NOT** need:

- GPU servers
- Training deep learning models
- Expensive third-party APIs
- Continuous video storage
- Proctoring browser extensions (for MVP)

This keeps the project:

- Feasible
 - Ethical
 - Startup-realistic
 - MCA-appropriate
-

7. FINAL SUMMARY (MEMORIZE THIS)

ProctoEase requires a React-based offline-first frontend, a FastAPI asynchronous backend, browser-based AI using MediaPipe, secure sandboxed code execution via Judge0, structural plagiarism detection using JPlag and MinHash, PostgreSQL with row-level security for multi-tenancy, and an explainable, human-in-the-loop proctoring model that prioritizes privacy, fairness, scalability, and sustainability.

Perfect — I'll give you a **clean, actionable, module-wise project plan, adjusted for a team of 3**, so that:

- You can **start coding immediately**
- Work is **parallel (no blocking)**
- The plan is **flexible** (easy to add features later)
- It is **academically strong + startup-realistic**

I'll structure this like a **real engineering execution plan**, not theory.

PROCTOEASE – MODULE-WISE PROJECT PLAN (TEAM OF 3)

TEAM STRUCTURE (VERY IMPORTANT)

We will **not** split randomly. We split by **system boundaries**.

Member 1 – Frontend + Edge AI Lead

Role: Candidate experience, proctoring logic, offline-first UX

Primary Stack: React, Browser APIs, MediaPipe

Member 2 – Backend + Security Lead

Role: APIs, authentication, multi-tenancy, business logic

Primary Stack: FastAPI, PostgreSQL, JWT, RBAC

Member 3 – Evaluation + DevOps Lead

Role: Code execution, plagiarism, async jobs, deployment

Primary Stack: Judge0, JPlag, Celery, Docker

You will collaborate, but **ownership is clear** — examiners love this.

HIGH-LEVEL PHASES (WHAT COMES FIRST, WHAT COMES LATER)

We will work in **6 phases**, each producing something demo-able.

Phase	Goal	Outcome
Phase 0	Setup & alignment	Repo + architecture ready
Phase 1	Core exam system	Exam creation & attempt
Phase 2	Proctoring MVP	AI-assisted monitoring
Phase 3	Evaluation engines	Code exec + plagiarism
Phase 4	Explainability + polish	XAI dashboards
Phase 5	Hardening & scale	Security, CI/CD

PHASE 0 – FOUNDATION (Week 1)

Goal

Everyone can run the project locally.

Common Tasks (ALL 3)

- Finalize tech stack (already done)
- Create **monorepo**
- Define coding standards
- Agree on API contracts

Repo Structure

```
proctoease/  
|— frontend/      # React  
|— backend/       # FastAPI  
|— workers/       # Celery tasks
```

```
|— judge0/          # Judge0 config
|— infra/           # Docker, CI/CD
|— docs/            # SRS, diagrams
```

Deliverable

- ✓ Docker Compose runs
 - ✓ Empty frontend + backend talking
-

PHASE 1 – CORE EXAM SYSTEM (Weeks 2–3)

MODULE 1: Authentication & Multi-Tenancy

Member 2 (Backend Lead)

- JWT auth (login, refresh)
- Tenant model
- Role-based access (Student / Recruiter / Admin)
- PostgreSQL RLS policies

APIs

- `/auth/login`
- `/auth/refresh`
- `/users`
- `/tenants`

DB Tables

- `users`

- tenants
 - roles
 - user_roles
-

MODULE 2: Exam Management

Member 2 (Backend)

- Create exam
- Add questions
- Time limits
- Publish exam

Member 1 (Frontend)

- Recruiter dashboard
- Exam creation UI
- Candidate exam list

Deliverable

- ✓ Candidate can log in
 - ✓ Recruiter can create exam
 - ✓ Candidate can start exam
-

PHASE 2 – AI-ASSISTED PROCTORING (Weeks 4–5)

MODULE 3: Browser-Based Proctoring (MVP)

Member 1 (Frontend + AI Lead)

Features

- Webcam access
- MediaPipe face detection
- Face present / missing
- Multiple face detection
- Page Visibility (tab switch)
- Audio level spike detection

Output (Important)

You **DO NOT** send video.

You send **events**:

```
{  
  "event": "TAB_SWITCH",  
  "timestamp": "2026-03-01T10:15:32",  
  "severity": "HIGH"  
}
```

Member 2 (Backend)

- Proctor event API
- Store events per exam session
- No judgment yet (just logging)

Deliverable

- ✓ Live proctoring telemetry
 - ✓ Events stored correctly
-

PHASE 3 – EVALUATION ENGINES

(Weeks 6–7)

MODULE 4: Code Execution (Judge0)

 **Member 3 (Evaluation Lead)**

Setup

- Self-host Judge0 (Docker)
- Secure network bridge to backend

Features

- Code submission
- Language selection
- Hidden test cases
- Time & memory limits

Flow

Frontend → Backend → Judge0 → Backend → Result

MODULE 5: Plagiarism Detection

 **Member 3**

Engines

- MinHash → fast clustering
- JPlag → deep structural match

Output

- Similarity percentage

- Highlighted matching blocks

Deliverable

- ✓ Coding exam fully evaluable
 - ✓ Plagiarism report generated
-

PHASE 4 – EXPLAINABLE AI & DASHBOARDS (Weeks 8–9)

MODULE 6: Heuristic Risk Scoring (XAI)

Member 2 (Backend)

- Weighted scoring engine
- Configurable thresholds

Tab switch	+20
Multiple faces	+35
Face missing	+15
Audio spike	+5

MODULE 7: Evidence Timeline UI

Member 1 (Frontend)

UI Features

- Timeline (time → events)
- Color-coded severity
- Click event → snapshot + explanation

This is your WOW feature

PHASE 5 – OFFLINE, SECURITY & DEPLOYMENT (Weeks 10–12)

MODULE 8: Offline-First Support

Member 1

- Service Worker
- IndexedDB storage
- Background sync

Member 2

- Idempotent APIs
- Safe replays

MODULE 9: Security & Compliance

Member 2

- Consent flows
- Data retention jobs
- Audit logs

MODULE 10: CI/CD & Deployment

Member 3

- Docker optimization
 - GitHub Actions
 - One-click deploy
-

HOW WE ADD FEATURES LATER (IMPORTANT)

Your architecture is **extensible by design**.

Easy Add-Ons Later

- Mobile app (React Native)
- AI bias audit reports
- ATS integration
- Interview scheduling
- Gaze heatmaps
- Screen recording (optional)

Why This Is Safe

- Event-based proctoring
 - Microservices
 - Async workers
 - No tight coupling
-

WHO PRESENTS WHAT (VIVA STRATEGY)

Member	Focus
You	Architecture + AI + XAI
Member 2	Backend, security, multi-tenancy
Member 3	Judge0, plagiarism, DevOps

This shows **clear ownership + teamwork**.

FINAL EXECUTION ADVICE (IMPORTANT)

- **Do not overbuild in Phase 1**
- Get a **working MVP fast**
- Add intelligence gradually
- Always keep:

“AI assists humans, not judges them”